Report #4: Implementing and Validating Data-at-Rest Encryption in OpenStack

Bryan Daniel, Justin Cockrell, Dylan Hagy
University of South Carolina Aiken
CSCI A592-PC
Dr. Ali AlSabeh
Spring 2026

Scenario

Following identity hardening, network segmentation and storage remediation, and governance enforcement, Southriver Software Solutions now operates a segmented and resource-governed OpenStack private cloud.

However, block storage volumes remain unencrypted at rest. This lab implements and validates encrypted block storage using native Cinder LUKS encryption within the existing DevStack deployment.

Prerequisites

- Persistent DevStack environment
- Functional Cinder LVM backend
- DEV project with CirrOS instance (DEV-APP)
- Admin and DEV OpenRC files
- Root access to OpenStack host

Learning Objectives

- Configure encrypted Cinder volume types [1]
- Provision encrypted volumes
- Validate LUKS enforcement at host level
- Compare encrypted vs unencrypted volumes
- Demonstrate protection against raw block inspection

Background

OpenStack Block Storage (Cinder) supports volume encryption using Linux Unified Key Setup (LUKS). When encryption is enabled on a volume type, Cinder performs encryption operations before data is written to disk [1]. This encryption workflow operates within the Cinder volume service and backend driver path as defined by the Block Storage Service architecture documentation [4].

Encrypted volumes rely on:

- Cinder volume types [2]

- Encryption provider configuration
- Key management integration (DevStack uses ConfKey internally)
- LVM-backed block devices [3]

These dependencies align with the documented Cinder block storage service architecture, which separates API, scheduler, and volume services while relying on backend drivers (such as LVM) to provision host-level block devices for tenant consumption [4].

Without encryption, raw block inspection of storage devices may reveal tenant data directly. Volume encryption protects against:

- Physical disk theft
- Snapshot-level disk inspection
- Hypervisor-level LVM mapping exposure
- Direct device reads via /dev/mapper

While previous reports hardened identity and network boundaries, confidentiality at the storage layer remained unaddressed. Cinder volume encryption is documented within the OpenStack Block Storage Administration Guide and associated configuration documentation [1] [2] [3].

Section 1: Baseline – Provisioning and Inspecting an Unencrypted Volume

Purpose

This section establishes a baseline for understanding the security risk posed by unencrypted block storage in OpenStack environments. By provisioning a standard unencrypted Cinder volume and attaching it to a tenant instance, we demonstrate that data written within a virtual machine remains accessible at the hypervisor level through direct block device inspection. This baseline configuration mirrors the default behavior in many OpenStack deployments where volume encryption has not been explicitly enabled.

The demonstration proceeds through four critical phases: authentication and volume creation, attachment to a running instance, writing identifiable test data from within the guest operating system, and finally, raw device inspection from the OpenStack host. The ability to retrieve plaintext data directly from the underlying logical volume device

illustrates a fundamental vulnerability: host-level access bypasses all tenant-level security controls, exposing confidential data to potential compromise through physical disk theft, snapshot analysis, or unauthorized hypervisor access.

This baseline not only quantifies the current risk but also provides a measurable point of comparison for validating the encryption controls implemented in subsequent sections.

1.1 Authenticate as Admin

Step 1: Authenticate using administrative credentials. Admin scope is required for volume type management and host-level inspection performed later in this section.

Command: cd ~/Desktop

Command: source admin-openrc.sh

Command: openstack token issue

```
ubuntu@ubuntu:~$ cd ~/Desktop
ubuntu@ubuntu:~/Desktop$ source admin-openrc.sh
Please enter your OpenStack Password for project admin as user admin:
ubuntu@ubuntu:~/Desktop$ openstack token issue
+------------+-----------------------------------------------------------------------------------------------------+
| Field      | Value                                                                                               |
+------------+-----------------------------------------------------------------------------------------------------+
| expires    | 2026-03-05T15:21:41+0000                                                                            |
| id         | gAAAAABpqZF1nJQLo6JFYFg4rxuYsF0-                                                                    |
|            | xJNqT7xyBwkZbYOZZhJhDvWGz6O1lGQN4krHoUqSvX2EPczjXeNiDS03aJw9RELMrriMJYkYomYREn-V-                                |
|            | hpfgs7ga1iHWUvi2BXUvYhR2SrjcEPiFanf0bvNdAohBiO66BT2TUWBOuudLYtUYPubIe0                                          |
| project_id | bd755daf91444d328d3f25316317a883                                                                            |
| user_id    | 2eee114ca75f4851850324b95989f70f                                                                            |
+------------+-----------------------------------------------------------------------------------------------------+
ubuntu@ubuntu:~/Desktop$
```

Expected Output: Token details showing you're authenticated

1.2 Create Unencrypted Volume (DEV)

Step 2: Switch to DEV user context. Scoping to the DEV project ensures the unencrypted volume is created within the correct tenant for attachment to `DEV-APP`.

Command: source dev-openrc.sh

Command: openstack volume create –size 1 dev-plain-volume

```
ubuntu@ubuntu:~/Desktop$ source dev-openrc.sh
Please enter your OpenStack Password for project dev as user DevUser1:
ubuntu@ubuntu:~/Desktop$ openstack volume create --size 1 dev-plain-volume
+---------------------------------+--------------------------------------+
| Field                           | Value                                |
+---------------------------------+--------------------------------------+
| attachments                     | []                                   |
| availability_zone               | nova                                 |
| bootable                        | False                                |
| cluster_name                    | None                                 |
| consumes_quota                  | True                                 |
| created_at                      | 2026-03-05T14:32:38.260638           |
| description                     | None                                 |
| encrypted                       | False                                |
| group_id                        | None                                 |
| id                              | 3119b330-5d9e-4d73-a68c-5b1c8d775730 |
| multiattach                     | False                                |
| name                            | dev-plain-volume                     |
| os-vol-host-attr:host           | None                                 |
| os-vol-mig-status-attr:migstat  | None                                 |
| os-vol-mig-status-attr:name_id  | None                                 |
| os-vol-tenant-attr:tenant_id    | None                                 |
| properties                      |                                      |
| provider_id                     | None                                 |
| replication_status              | None                                 |
| service_uuid                    | None                                 |
| shared_targets                  | True                                 |
| size                            | 1                                    |
| snapshot_id                     | None                                 |
| source_volid                    | None                                 |
| status                          | creating                             |
| type                            | lvmdriver-1                          |
```

Step 3: Wait a moment, then verify. Confirming the volume reaches `available` status before attachment prevents errors caused by attempting to attach a volume that has not finished initializing.

Command openstack volume show dev-plain-volume

```
ubuntu@ubuntu:~/Desktop$ openstack volume show dev-plain-volume
+--------------------------------+--------------------------------------+
| Field                          | Value                                |
+--------------------------------+--------------------------------------+
| attachments                    | []                                   |
| availability_zone              | nova                                 |
| bootable                       | False                                |
| cluster_name                   | None                                 |
| consumes_quota                 | True                                 |
| created_at                     | 2026-03-05T14:32:38.000000           |
| description                    | None                                 |
| encrypted                      | False                                |
| group_id                       | None                                 |
| id                             | 3119b330-5d9e-4d73-a68c-5b1c8d775730 |
| multiattach                    | False                                |
| name                           | dev-plain-volume                     |
| os-vol-host-attr:host          | None                                 |
| os-vol-mig-status-attr:migstat | None                                 |
| os-vol-mig-status-attr:name_id | None                                 |
| os-vol-tenant-attr:tenant_id   | 4f97f0eebec74d8b83b9296bc70dd0bb     |
| properties                     |                                      |
| provider_id                    | None                                 |
| replication_status             | None                                 |
| service_uuid                   | 591a48d7-86cc-49ba-a5c0-bcfc82bf953c |
| shared_targets                 | False                                |
| size                           | 1                                    |
| snapshot_id                    | None                                 |
| source_volid                   | None                                 |
| status                         | available                            |
| type                           | lvmdriver-1                          |
```

Expected Output: Status should be "available"

Step 4: Attach the volume to `DEV-APP`. Attaching the unencrypted volume to the running instance makes it accessible as a block device from within the guest OS.
Command: openstack server add volume DEV-APP dev-plain-volume

```
ubuntu@ubuntu:~/Desktop$ openstack server add volume DEV-APP dev-plain-volume
+-----------------------+--------------------------------------+
| Field                 | Value                                |
+-----------------------+--------------------------------------+
| ID                    | 3119b330-5d9e-4d73-a68c-5b1c8d775730 |
| Server ID             | e307b644-e163-40c6-b90e-c583e5ff1055 |
| Volume ID             | 3119b330-5d9e-4d73-a68c-5b1c8d775730 |
| Device                | /dev/vdb                             |
| Tag                   | None                                 |
| Delete On Termination | False                                |
+-----------------------+--------------------------------------+
```

Expected Output: Should show attached to DEV-APP with device path /dev/vdb

Step 5: Wait a moment, then verify. Confirming the attachment before accessing the console ensures the device path is stable and visible inside the guest.

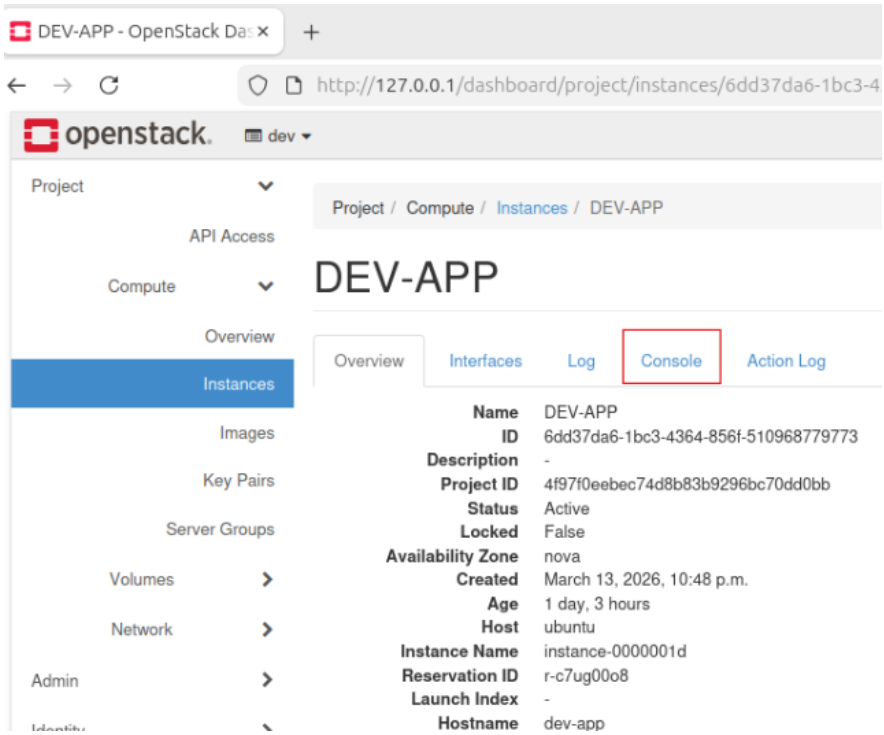
Command: openstack volume show dev-plain-volume

```
ubuntu@ubuntu:~/Desktop$ openstack volume show dev-plain-volume
+--------------------------------+--------------------------------------+
| Field                          | Value                                |
+--------------------------------+--------------------------------------+
| attachments                    | []                                   |
| availability_zone              | nova                                 |
| bootable                       | False                                |
| cluster_name                   | None                                 |
| consumes_quota                 | True                                 |
| created_at                     | 2026-03-05T14:32:38.000000           |
| description                    | None                                 |
| encrypted                      | False                                |
| group_id                       | None                                 |
| id                             | 3119b330-5d9e-4d73-a68c-5b1c8d775730 |
| multiattach                    | False                                |
| name                           | dev-plain-volume                     |
| os-vol-host-attr:host          | None                                 |
| os-vol-mig-status-attr:migstat | None                                 |
| os-vol-mig-status-attr:name_id | None                                 |
| os-vol-tenant-attr:tenant_id   | 4f97f0eebec74d8b83b9296bc70dd0bb     |
| properties                     |                                      |
| provider_id                    | None                                 |
| replication_status             | None                                 |
| service_uuid                   | 591a48d7-86cc-49ba-a5c0-bcfc82bf953c |
| shared_targets                 | False                                |
| size                           | 1                                    |
| snapshot_id                    | None                                 |
| source_volid                   | None                                 |
| status                         | available                            |
```

1.3 Write Test Data from CirrOS

Step 6: Access the `DEV-APP` console via Horizon. Using the browser console provides direct guest OS access without requiring SSH key setup or floating IP configuration.

1. Login to Horizon (http://<your-ip>/dashboard)
2. Navigate to Project → Compute → Instances
3. Click on DEV-APP instance
4. Click "Console" tab



Step 7: Check available block devices. Confirming `/dev/vdb` is visible inside the guest verifies the volume attachment completed successfully before formatting.

```
lsblk
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
vda     252:0    0    1G  0 disk
|-vda1  252:1    0 1015M  0 part /
`-vda15 252:15   0    8M  0 part
vdb     252:16   0    1G  0 disk
$
```

You should see /dev/vdb (the attached volume).

Step 8: Format and mount the volume. Formatting creates a filesystem on the raw block device and mounting makes it writable from within the guest OS.

Command: sudo mount /dev/vdb /mnt

```
$ sudo mkfs.ext4 /dev/vdb
mke2fs 1.46.5 (30-Dec-2021)
Discarding device blocks: done
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: 0af0b5d2-44c7-4a12-8f96-890d4c73b9aa
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done

$ _
```

```
$ sudo mount /dev/vdb /mnt
$ _
```

Step 9: Verify the data was written. Confirming the test string is readable from within the guest establishes that the write succeeded before moving to host-level inspection.

Command: echo “UNENCRYPTED_TEST_DATA” | sudo tee /mnt/test.txt

Command: cat /mnt/test.txt

```
$ echo "UNENCRYPTED_TEST_DATA" | sudo tee /mnt/test.txt
UNENCRYPTED_TEST_DATA
$ cat /mnt/test.txt
UNENCRYPTED_TEST_DATA
$
```

Expected Output: UNENCRYPTED_TEST_DATA

1.4 Inspect Raw Device on Host

On the OpenStack host (actual terminal - not the CirrOS VM):

Step 10: List logical volumes. Identifying the LVM logical volume that corresponds to the Cinder volume confirms the host-level device path needed for raw inspection.

Command: sudo lvs

```
ubuntu@ubuntu:~/Desktop$ sudo lvs
  LV                                          VG                        Attr       LSize  Pool
 Origin                                      Data%  Meta%  Move Log Cpy%Sync Convert
  stack-volumes-lvmdriver-1-pool              stack-volumes-lvmdriver-1 twi-aotz-- 28.50g
                                              1.56   11.17
  volume-26e49a1f-f36f-4f25-8275-56590d06bb75 stack-volumes-lvmdriver-1 Vwi-a-tz--  1.00g stack-volumes-lvmdriver-1-pool
                                              0.00
  volume-286e18b0-551e-4b8f-b6af-0c3a9b60405a stack-volumes-lvmdriver-1 Vwi-a-tz--  1.00g stack-volumes-lvmdriver-1-pool
 volume-cf54859a-3c02-40ab-b02a-80c8894eefab 10.94
  volume-3119b330-5d9e-4d73-a68c-5b1c8d775730 stack-volumes-lvmdriver-1 Vwi-aotz--  1.00g stack-volumes-lvmdriver-1-pool
                                              4.79
  volume-7e8ff32d-8216-4f90-a37e-7cfe4559ba2f stack-volumes-lvmdriver-1 Vwi-aotz--  1.00g stack-volumes-lvmdriver-1-pool
 volume-286e18b0-551e-4b8f-b6af-0c3a9b60405a 15.26
  volume-93d1a3d7-c64c-4d4d-94d2-8bf2abed1016 stack-volumes-lvmdriver-1 Vwi-aotz--  1.00g stack-volumes-lvmdriver-1-pool
 volume-286e18b0-551e-4b8f-b6af-0c3a9b60405a 15.27
  volume-cf54859a-3c02-40ab-b02a-80c8894eefab stack-volumes-lvmdriver-1 Vwi-aotz--  1.00g stack-volumes-lvmdriver-1-pool
                                              15.26
  volume-ef5ba569-b679-4d23-8470-3b12ed119673 stack-volumes-lvmdriver-1 Vwi-a-tz--  2.00g stack-volumes-lvmdriver-1-pool
                                              0.00
  volume-f36a992b-94d3-456e-8dc6-8266193bd9d6 stack-volumes-lvmdriver-1 Vwi-aotz--  1.00g stack-volumes-lvmdriver-1-pool
 volume-286e18b0-551e-4b8f-b6af-0c3a9b60405a 15.26
ubuntu@ubuntu:~/Desktop$
```

The volume ID matches what was seen earlier: 3119b330-5d9e-4d73-a68c-5b1c8d775730

Step 11: List Block Devices :

Command: sudo lsblk

```
loop17                                                                  7:17   0    30G  0 loop
├─stack--volumes--lvmdriver--1-stack--volumes--lvmdriver--1--pool_tmeta     252:0    0    32M  0 lvm
│ └─stack--volumes--lvmdriver--1-stack--volumes--lvmdriver--1--pool-tpool   252:2    0  28.5G  0 lvm
│   ├─stack--volumes--lvmdriver--1-stack--volumes--lvmdriver--1--pool       252:3    0  28.5G  1 lvm
│   ├─stack--volumes--lvmdriver--1-volume--26e49a1f--f36f--4f25--8275--56590d06bb75
│   │                                                                      252:4    0     1G  0 lvm
│   ├─stack--volumes--lvmdriver--1-volume--cf54859a--3c02--40ab--b02a--80c8894eefab
│   │                                                                      252:5    0     1G  0 lvm
│   ├─stack--volumes--lvmdriver--1-volume--286e18b0--551e--4b8f--b6af--0c3a9b60405a
│   │                                                                      252:6    0     1G  0 lvm
│   ├─stack--volumes--lvmdriver--1-volume--93d1a3d7--c64c--4d4d--94d2--8bf2abed1016
│   │                                                                      252:7    0     1G  0 lvm
│   ├─stack--volumes--lvmdriver--1-volume--f36a992b--94d3--456e--8dc6--8266193bd9d6
│   │                                                                      252:8    0     1G  0 lvm
│   ├─stack--volumes--lvmdriver--1-volume--7e8ff32d--8216--4f90--a37e--7cfe4559ba2f
│   │                                                                      252:9    0     1G  0 lvm
│   ├─stack--volumes--lvmdriver--1-volume--ef5ba569--b679--4d23--8470--3b12ed119673
│   │                                                                      252:10   0     2G  0 lvm
│   └─stack--volumes--lvmdriver--1-volume--3119b330--5d9e--4d73--a68c--5b1c8d775730
│                                                                          252:11   0     1G  0 lvm
```

This confirms the device is visible at the host block layer and the block ID matches the volume provisioned through Cinder: `3119b330-5d9e-4d73-a68c-5b1c8d775730`.

Step 12: Attempt to read plaintext data directly from the block device. Reading the raw device with `strings` bypasses all guest OS access controls and directly retrieves any human-readable content stored on the volume.

Command: sudo strings /dev/mapper/stack--volumes—lvmdriver—1-volume—3119b330—5d9e—4d73—a68c—5b1c8d775730 | grep UNENCRYPTED_TEST_DATA

```
ubuntu@ubuntu:~/Desktop$ sudo strings /dev/mapper/stack--volumes--lvmdriver--1-volume--3119b330--5d9e--4d73--a68c--5b1c8
d775730 | grep UNENCRYPTED_TEST_DATA
UNENCRYPTED_TEST_DATA
ubuntu@ubuntu:~/Desktop$
```

Expected Output: You should see UNENCRYPTED_TEST_DATA printed

Section 1 Outcomes

- Successfully provisioned an unencrypted 1GB volume (dev-plain-volume) in the DEV project
- Attached the volume to DEV-APP instance as /dev/vdb
- Wrote test data (UNENCRYPTED_TEST_DATA) to the volume from within the CirrOS VM
- Demonstrated critical vulnerability: Plaintext data was directly readable from the hypervisor level using raw block device inspection
- Security implication confirmed: Host-level access bypasses all tenant-level security controls

Critical Finding:

The strings command successfully retrieved UNENCRYPTED_TEST_DATA from **/dev/mapper/stack--volumes--lvmdriver--1-volume--3119b330--5d9e--4d73--a68c--5b1c8d775730**, proving that:

- Tenant data is stored in plaintext on block devices
- Physical disk theft or hypervisor compromise exposes all data
- LVM mapping provides direct access to tenant storage without authentication

Section 2: Creating and Configuring an Encrypted Volume Type

Purpose

Having demonstrated the plaintext exposure vulnerability in Section 1, this section implements the cryptographic controls necessary to protect data at rest. OpenStack Cinder supports volume encryption through volume types through administrator-defined storage classes that specify backend properties and encryption parameters. By creating a dedicated encrypted volume type, administrators can enforce LUKS (Linux Unified Key Setup) encryption on all volumes provisioned from that type, ensuring that data is encrypted before being written to physical storage devices.

This section executes three critical administrative operations. First, we create a new volume type named 'encrypted-luks' to serve as a distinct storage class for secure volumes. Second, we associate this type with the LVM backend driver to ensure proper integration with the existing block storage infrastructure. Third, we configure the encryption metadata by specifying LUKS as the encryption provider, AES-XTS-Plain64 as the cipher suite, 256-bit key size for cryptographic strength, and front-end control location to position encryption operations at the nova-compute layer.

The successful configuration of this volume type establishes the policy framework that will govern all subsequent encrypted volume provisioning, transforming encryption from an optional feature into an enforced architectural control.

2.1 Return to Admin Scope

Step 1: Return to admin. Administrative scope is required for all volume type and encryption configuration operations that follow.

Command: source admin-openrc.sh

```
ubuntu@ubuntu:~/Desktop$ source admin-openrc.sh
Please enter your OpenStack Password for project admin as user admin:
ubuntu@ubuntu:~/Desktop$
```

2.2 Create the Encrypted Volume Type

Step 2: Create the encrypted volume type. A named volume type creates a distinct storage class in Cinder that tenants and administrators can reference when provisioning volumes that must be encrypted.

Command: openstack volume type create encrypted-luks

```
ubuntu@ubuntu:~/Desktop$ openstack volume type create encrypted-luks
+-------------+--------------------------------------+
| Field       | Value                                |
+-------------+--------------------------------------+
| description | None                                 |
| id          | 817d87c4-ec8d-486e-b164-078bcd520715 |
| is_public   | True                                 |
| name        | encrypted-luks                       |
+-------------+--------------------------------------+
ubuntu@ubuntu:~/Desktop$
```

This shows successful creation of the encrypted-luks volume.

Step 3: Assign LVM backend. Binding the volume type to the LVM backend ensures that encrypted volumes are provisioned through the same backend driver used for standard volumes, maintaining infrastructure consistency.

Command: openstack volume type set encrypted-luks –property volume_backend_name=lvmdriver-1

```
+-------------+--------------------------------------+
ubuntu@ubuntu:~/Desktop$ openstack volume type set encrypted-luks --property volume_backend_name=lvmdriver-1
ubuntu@ubuntu:~/Desktop$
```

This shows successful assignment of the 'encrypted-luks' volume.

Step 4: Configure LUKS encryption. This step attaches the cryptographic policy to the volume type — specifying the LUKS provider, `aes-xts-plain64` cipher, 256-bit key size, and `front-end` control location — so that every volume created from this type is automatically encrypted.

Command: openstack volume type set encrypted-luks –encryption-provider luks –encryption-cipher aes-xts –plain64 –encryption-key-size 256 –encryption-control-location front-end

```
ubuntu@ubuntu:~/Desktop$ openstack volume type set encrypted-luks --encryption-provider luks --encryption-cipher aes-xts
-plain64 --encryption-key-size 256 --encryption-control-location front-end
No existing encryption type found, creating new encryption type for this volume type ...
ubuntu@ubuntu:~/Desktop$
```

This shows successful configuration of the 'encrypted-luks' volume.

Step 5: Verify configuration. Confirming the encryption metadata is correctly stored in the Cinder database before provisioning any encrypted volumes prevents silent misconfiguration.

Command: openstack volume type show encrypted-luks

```
ubuntu@ubuntu:~/Desktop$ openstack volume type show encrypted-luks
+--------------------+--------------------------------------+
| Field              | Value                                |
+--------------------+--------------------------------------+
| access_project_ids | None                                 |
| description        | None                                 |
| id                 | 817d87c4-ec8d-486e-b164-078bcd520715 |
| is_public          | True                                 |
| name               | encrypted-luks                       |
| properties         | volume_backend_name='LVM'            |
| qos_specs_id       | None                                 |
+--------------------+--------------------------------------+
ubuntu@ubuntu:~/Desktop$
```

This shows successful creation of the 'encrypted-luks' volume.

Command: openstack volume type show encrypted-luks

```
ubuntu@ubuntu:~/Desktop$ openstack volume type list --encryption-type
+--------------------------------------+----------------+-----------+-----------------------------------------+
| ID                                   | Name           | Is Public | Encryption                              |
+--------------------------------------+----------------+-----------+-----------------------------------------+
| 817d87c4-ec8d-486e-b164-078bcd520715 | encrypted-luks | True      | cipher='aes-xts-plain64',               |
|                                      |                |           | control_location='front-end', encryption_i |
|                                      |                |           | d='2704aa32-ce3c-4d7a-ad0d-86b6d8534763',  |
|                                      |                |           | key_size='256', provider='luks'         |
| 0b9b5340-5be6-45cb-b85e-d613e570a9a0 | lvmdriver-1    | True      | -                                       |
| fb425764-4fc3-4b0c-8b4b-73cb730a33d1 | __DEFAULT__    | True      | -                                       |
+--------------------------------------+----------------+-----------+-----------------------------------------+
ubuntu@ubuntu:~/Desktop$
```

This shows successful creation of the 'encrypted-luks' encryption properties in the output.

Section 2 Outcomes

- Successfully created encrypted volume type ('encrypted-luks') with admin privileges

- Assigned LVM backend to the encrypted volume type using 'backend_name=LVM property ‘
- Configured LUKS encryption parameters:
  - Provider: ‘luks’
  - Cipher: ‘aes-xts-plain64' (industry-standard AES encryption with XTS mode)
  - Key size: 256 bits
  - Control location: ‘front-end’ (encryption handled at nova-compute level)
- Verified encryption configuration using ‘openstack volume type list --encryption-type ‘
- Encryption metadata successfully stored in Cinder database for future volume provisioning

Section 3: Provisioning and Validating an Encrypted Volume

Purpose

Following the configuration of the encrypted volume type in Section 2, this section validates that the defined encryption policy is actively enforced during volume provisioning. OpenStack Cinder supports volume encryption through administrator-defined volume types that apply cryptographic controls at the block storage layer using Linux Unified Key Setup (LUKS) [1]. To confirm that these controls function as intended, an encrypted volume is created within the DEV project and attached to the existing ‘DEV-APP’ instance. This step ensures that the encryption configuration previously established is applied automatically during normal tenant storage operations.

After provisioning the encrypted volume, identifiable test data is written from within the guest operating system to simulate a typical tenant workload. The OpenStack host is then used to inspect the underlying block device directly using the same raw inspection technique demonstrated in Section 1. Unlike the baseline scenario, plaintext data should not be visible when examining the encrypted block device. This comparison verifies that LUKS encryption is correctly protecting tenant data at the storage layer and mitigating the plaintext exposure vulnerability previously identified.

3.1 Create Encrypted Volume (DEV)

Step 1: Switch to DEV user context. Scoping to the DEV project ensures the encrypted volume is created and owned by the correct tenant.

Command: cd Desktop

Command: source dev-openrc.sh

```
ubuntu@ubuntu:~$ cd Desktop
ubuntu@ubuntu:~/Desktop$ source dev-openrc.sh
Please enter your OpenStack Password for project dev as user DevUser1:

ubuntu@ubuntu:~/Desktop$
```

Step 2: Create a new encrypted volume using the `encrypted-luks` volume type. Specifying the encrypted volume type at creation time triggers Cinder to apply the LUKS encryption policy to the provisioned block device.

Command: openstack volume create –size 1 –type encrypted-luks dev-secure-volume

```
ubuntu@ubuntu:~/Desktop$ openstack volume create --size 1 --type encrypted-luks dev-secure-volume
+---------------------------------+--------------------------------------+
| Field                           | Value                                |
+---------------------------------+--------------------------------------+
| attachments                     | []                                   |
| availability_zone               | nova                                 |
| bootable                        | False                                |
| cluster_name                    | None                                 |
| consumes_quota                  | True                                 |
| created_at                      | 2026-03-12T23:53:48.164012           |
| description                     | None                                 |
| encrypted                       | False                                |
| group_id                        | None                                 |
| id                              | a20ab0c9-5fcb-4097-ad02-71ab8f34c08d |
| multiattach                     | False                                |
| name                            | dev-secure-volume                    |
| os-vol-host-attr:host           | None                                 |
| os-vol-mig-status-attr:migstat  | None                                 |
| os-vol-mig-status-attr:name_id  | None                                 |
| os-vol-tenant-attr:tenant_id    | None                                 |
| properties                      |                                      |
| provider_id                     | None                                 |
| replication_status              | None                                 |
| service_uuid                    | None                                 |
| shared_targets                  | True                                 |
| size                            | 1                                    |
| snapshot_id                     | None                                 |
| source_volid                    | None                                 |
| status                          | creating                             |
| type                            | encrypted-luks                       |
```

Step 3: Verify that the encrypted volume was created successfully. Volume status should display `available` and the `encrypted-luks` volume type, confirming the encryption configuration was applied.

Command: openstack volume show dev-secure-volume

```
ubuntu@ubuntu:~/Desktop$ openstack volume show dev-secure-volume
+--------------------------------+--------------------------------------+
| Field                          | Value                                |
+--------------------------------+--------------------------------------+
| attachments                    | []                                   |
| availability_zone              | nova                                 |
| bootable                       | False                                |
| cluster_name                   | None                                 |
| consumes_quota                 | True                                 |
| created_at                     | 2026-03-12T23:53:48.000000           |
| description                    | None                                 |
| encrypted                      | False                                |
| group_id                       | None                                 |
| id                             | a20ab0c9-5fcb-4097-ad02-71ab8f34c08d |
| multiattach                    | False                                |
| name                           | dev-secure-volume                    |
| os-vol-host-attr:host          | None                                 |
| os-vol-mig-status-attr:migstat | None                                 |
| os-vol-mig-status-attr:name_id | None                                 |
| os-vol-tenant-attr:tenant_id   | 4f97f0eebec74d8b83b9296bc70dd0bb     |
| properties                     |                                      |
| provider_id                    | None                                 |
| replication_status             | None                                 |
| service_uuid                   | 591a48d7-86cc-49ba-a5c0-bcfc82bf953c |
| shared_targets                 | False                                |
| size                           | 1                                    |
| snapshot_id                    | None                                 |
| source_volid                   | None                                 |
| status                         | available                            |
| type                           | encrypted-luks                       |
| updated_at                     | 2026-03-12T23:53:49.000000           |
| user_id                        | f25bd4eea4034da9b65d156c48a50cd2     |
```

Volume status should display available and the encrypted-luks volume type.

Step 4: Attach the encrypted volume to the `DEV-APP` instance. The volume should appear attached to `DEV-APP` with an assigned device path, making it accessible from within the guest OS.

Command: openstack server add volume DEV-APP dev-secure-volume

```
ubuntu@ubuntu:/opt/stack$ openstack server add volume DEV-APP dev-secure-volume
+-----------------------+--------------------------------------+
| Field                 | Value                                |
+-----------------------+--------------------------------------+
| ID                    | a20ab0c9-5fcb-4097-ad02-71ab8f34c08d |
| Server ID             | e307b644-e163-40c6-b90e-c583e5ff1055 |
| Volume ID             | a20ab0c9-5fcb-4097-ad02-71ab8f34c08d |
| Device                | /dev/vdc                             |
| Tag                   | None                                 |
| Delete On Termination | False                                |
+-----------------------+--------------------------------------+
```

The volume should appear attached to DEV-APP with an assigned device path.

3.2 Write Data to Encrypted Volume

Inside the DEV-APP instance console:

Step 5: Identify attached block devices:

Command: lsblk

```
NAME    MAJ:MIN RM  SIZE RO TYPE
vda     252:0    0    1G  0 disk
|-vda1  252:1    0 1015M  0 part
`-vda15 252:15   0    8M  0 part
vdb     252:16   0    1G  0 disk
vdd     252:48   0    1G  0 disk
```

The encrypted volume should appear as a new device (for example `/dev/vdd`), distinguishable from the unencrypted volume attached in Section 1.

Step 6: Format the encrypted volume with an ext4 filesystem. Creating a filesystem on the encrypted device initializes it for use while LUKS handles encryption transparently beneath the filesystem layer.

Command: sudo mkfs.ext4 /dev/vdd

```
vdd     252:48   0    1G  0 disk
$ sudo mkfs.ext4 /dev/vdd
mke2fs 1.46.5 (30-Dec-2021)
Discarding device blocks: done
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: 4f81dfca-e108-41aa-827c-13eaf89e8945
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
```

Step 7: Mount the encrypted volume to the filesystem. Mounting makes the encrypted volume writable from within the guest OS at a chosen mount point.

Command: sudo mount /dev/vdd /mnt

```
$ sudo mount /dev/vdd /mnt
$
```

Step 8: Write test data to the encrypted volume and flush filesystem buffers to ensure the data is written to disk. The file `secure.txt` should contain the string `ENCRYPTED_TEST_DATA`, providing a known value to look for during host-level inspection.

Command: lsblk
Command: sudo mkfs.ext4 /dev/vdd
Command: sudo mount /dev/vdd /mnt
Command: echo “ENCRYPTED_TEST_DATA” | sudo tee /mnt/secure.txt
Command: sync

```
dev-app login: cirros
Password:
$ lsblk
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
vda     252:0    0    1G  0 disk
|-vda1  252:1    0 1015M  0 part /
`-vda15 252:15   0    8M  0 part
vdb     252:16   0    1G  0 disk
vdc     252:32   0    1G  0 disk
vdd     252:48   0 1022M  0 disk
$ sudo mkfs.ext4 /dev/vdd
mke2fs 1.46.5 (30-Dec-2021)
Discarding device blocks: done
Creating filesystem with 261632 4k blocks and 65408 inodes
Filesystem UUID: 24976300-1df1-4277-a5e6-1092417a25fe
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

$ sudo mount /dev/vdd /mnt
$ echo "ENCRYPTED_TEST_DATA" | sudo tee /mnt/secure.txt
ENCRYPTED_TEST_DATA
$ sync
$
```

The file secure.txt should contain the string ENCRYPTED_TEST_DATA.

3.3 Validate Encryption on Host

On the OpenStack host system:

Step 9: List mapped block devices to identify the encrypted volume. A `crypt`-mapped device in the output confirms that LUKS is active and the volume is being presented to the guest through an encryption layer.

Command: ls /dev/mapper

```
ubuntu@ubuntu:~/Desktop$ ls /dev/mapper
control
stack--volumes--lvmdriver--1-stack--volumes--lvmdriver--1--pool
stack--volumes--lvmdriver--1-stack--volumes--lvmdriver--1--pool_tdata
stack--volumes--lvmdriver--1-stack--volumes--lvmdriver--1--pool_tmeta
stack--volumes--lvmdriver--1-stack--volumes--lvmdriver--1--pool-tpool
stack--volumes--lvmdriver--1-volume--0a3f671e--1aec--472e--a380--7a959c85b008
```

You should see a crypt-mapped device corresponding to the encrypted volume.

Step 10: Attempt raw inspection of the encrypted block device. No plaintext output should appear, confirming that the data is encrypted at the block layer and that the same `strings`-based inspection technique that exposed data in Section 1 is no longer effective.

Command: sudo string /dev/mapper/stack--volumes—lmdriver—1-volume-0a3f671e—1aec—472e—a380—7a959c85b00b | grep ENCRYPTED_TEST_DATA

```
ubuntu@ubuntu:~/Desktop$ sudo strings /dev/mapper/stack--volumes--lvmdriver--1-volume--0a3f671e--1aec--472e--a380--7a959
c85b008 | grep ENCRYPTED_TEST_DATA
ubuntu@ubuntu:~/Desktop$
```

No plaintext output should appear, confirming that the data is encrypted at the block layer.

Engagement Summary

This lab demonstrates that:

- Unencrypted volumes expose plaintext through raw block inspection
- Encrypted volumes prevent readable data exposure
- LUKS encryption is enforced at the block layer
- Encryption remains transparent to tenant workloads
- No additional services beyond native Cinder were required

Encrypted block storage now complements:

- RBAC enforcement
- Network isolation

- Quota governance

References

[1] OpenStack Foundation, “Volume Encryption,” OpenStack Block Storage Administration Guide, 2024. [Online]. Available: https://docs.openstack.org/cinder/latest/admin/volume-encryption.html

[2] OpenStack Foundation, “Volume Types,” OpenStack Block Storage Administration Guide, 2024. [Online]. Available: https://docs.openstack.org/cinder/latest/admin/volume-types.html

[3] OpenStack Foundation, “Configure the LVM Backend,” OpenStack Block Storage Configuration Guide, 2024. [Online]. Available: https://docs.openstack.org/cinder/latest/configuration/block-storage/drivers/lvm-volume-driver.html

[4] OpenStack Foundation, “Block Storage Service Architecture,” OpenStack Documentation, 2024. [Online]. Available: https://docs.openstack.org/cinder/latest/contributor/architecture.html